

<https://github.com/pudinh0/bootcamp4-6.git>

5

Kotlin Essentials: Más allá de lo básico

^

✓

Introducción

✓

Parejas

✓

Quiz: Tiempo de práctica

✓

Colecciones

✓

Quiz: Tiempo de práctica

✓

Constantes

✓

Quiz: Tiempo de práctica

✓

Funciones de extensión

✓

Quiz: Tiempo de práctica

✓

Clases genéricas

✓

Quiz: Tiempo de práctica

✓

Construcción del acuario

✓

Entrada y salida genérica

✓

Quiz: Tiempo de práctica

✓

Funciones genéricas

✓

Quiz: Tiempo de práctica

✓

Anotaciones

✓

Rupturas etiquetadas

✓

Resumen

4

Clases

^

✓

Introducción

✓

Definición de clases

✓

Quiz: Tiempo de práctica

✓

Constructores

✓

Quiz: Tiempo de práctica

✓

Construyendo el acuario

✓

Herencia

✓

Quiz: Tiempo de práctica

✓

Interfaces

✓

Quiz: Tiempo de práctica

✓

Clases de datos

✓

Quiz: Tiempo de práctica

✓

Clases de Propósito Especial

✓

Quiz: Tiempo de práctica

✓

Resumen

6

Manipulación funcional

^

✓

Introducción

✓

Resumen de Lambdas

✓

Quiz: Tiempo de práctica

✓

Funciones de orden superior

✓

Quiz: Tiempo de práctica

✓

Inlines

✓

Construyendo el acuario

✓

Quiz: Tiempo de práctica

✓

SAM

✓

Resumen

Lección

0

Completado

Clases

Aprende cómo funciona la programación orientada a objetos en Kotlin. Crea clases privadas y públicas, incluyendo clases abstractas e interfaces. Entiende las clases de datos y las clases de propósito especial.

▶

Lección

1

Completado

Kotlin Essentials: Más allá de lo básico

Aprende sobre características más avanzadas como pares, anotaciones y funciones de extensión. Entiende cómo usar constantes, colecciones y genéricos en Kotlin.

▶

Lección

2

Completado

Manipulación funcional

Sumergido en lambdas y funciones de orden superior en Kotlin. Aprende a escribir funciones en línea e interfaces de Método Abstracto Único (SAM).

▶ Currículum

```

package unidad5

3 Usages 2 Inheritors
open class MaterialBase(val cantidadNecesitada: Int = 1)

1 Usage
class Madera : MaterialBase(cantidadNecesitada = 4)

1 Usage
class Ladrillo : MaterialBase(cantidadNecesitada = 8)

// 2. Clase genérica con
2 Usages
class Building<T> : MaterialBase<(val material: T) {

    1 Usage
    val MaterialBase: Int = 100

    1 Usage
    val materialActualNecesitado: Int
    |   get() = MaterialBase * material.cantidadNecesitada

    2 Usages
    fun build() {
    |   println("$materialActualNecesitado ${material::class.simpleName} necesitado")
    |   }
}

// 3. Función principal de prueba
fun main() {
    val woodBuilding = Building(material = Madera())

```

```

kotlin run --module bootcamp4-6 --main-class unidad5.LibrosKt
✓ Compilation successful for 'bootcamp4-6' [jvm]
Here is your book Romeon and Juliet by William Shakespeare
Here is your book Romeon and Juliet by William Shakespeare written in 1597

Process finished with exit code 0

```

Pregunta del cuestionario

Sigamos con nuestra configuración de libros para practicar la creación de constantes en Kotlin. Hay 3 formas diferentes de crear constantes en Kotlin. No es el tema más emocionante, pero usarás constantes todo el tiempo.



¡Gracias por completarlo!

¡Bien hecho!

Continuar

Lista de tareas

```
// 3 y 4. Evalúa la entrada y llama a move() con la lambda adecuada
1 Usage
fun makeMove(direction: String?) {
    when (direction?.trim()?.lowercase()) {
        "w" -> move( where = north)
        "s" -> move( where = south)
        "d" -> move( where = east)
        "a" -> move( where = west)
        else -> move( where = end)
    }
}
```

```
// 2. Enum Directions
6 Usages
enum class Directions {
    1 Usage
    NORTH,
    1 Usage
    SOUTH,
    1 Usage
    EAST,
    1 Usage
    WEST,
    1 Usage
    START,
    1 Usage
    END
}
```

```
// 3. Clase Game
1 Usage
class Game {
    // 4. Lista mutable con START
    9 Usages
    var path = mutableListOf<Directions>().add(START)

    // 5. Lambdas que agregan la dirección correspondiente
    1 Usage
    val north = { path.add(Directions.NORTH) }
    1 Usage
    val south = { path.add(Directions.SOUTH) }
    1 Usage
    val east = { path.add(Directions.EAST) }
    1 Usage
    val west = { path.add(Directions.WEST) }

    // 6. Lambda end
    1 Usage
    val end = {
        path.add(Directions.END)
        println("Game Over: $path")
        path.clear()
        false
    }
}
```

```

// 7. Función principal
fun main() {
    // 8. Instancia de Game
    val game = Game()

    // 9. Pruebas solicitadas
    println(game.path)

    game.north()
    game.south()
    game.east()
    game.west()
    game.end()

    println(game.path)
}

```

```

Kotlin run --module bootcamp4-6 --main-class unidad6.DireccionesKt
✓ Compilation successful for 'bootcamp4-6' [jvm]
Enter a direction: a/s/w/d: w
Enter a direction: a/s/w/d: w
Enter a direction: a/s/w/d: w
Enter a direction: a/s/w/d: w
Enter a direction: a/s/w/d: ww
Game Over: [START, NORTH, NORTH, NORTH, NORTH, END]

Process finished with exit code 0

```

```

class Game {
    8 Usages
    var path = mutableListOf(Direcciones.START)

    // Lambdas de dirección (retornan Boolean)
    1 Usage
    val north = { path.add(Direcciones.NORTH) }
    1 Usage
    val south = { path.add(Direcciones.SOUTH) }
    1 Usage
    val east = { path.add(Direcciones.EAST) }
    1 Usage
    val west = { path.add(Direcciones.WEST) }

    1 Usage
    val end = {
        path.add(Direcciones.END)
        println("Game Over: $path")
        path.clear()
        false
    }

    // 1 y 2. Función de orden superior que invoca el lambda recibido
    5 Usages
    fun move(where: () -> Boolean) {
        where()
    }
}

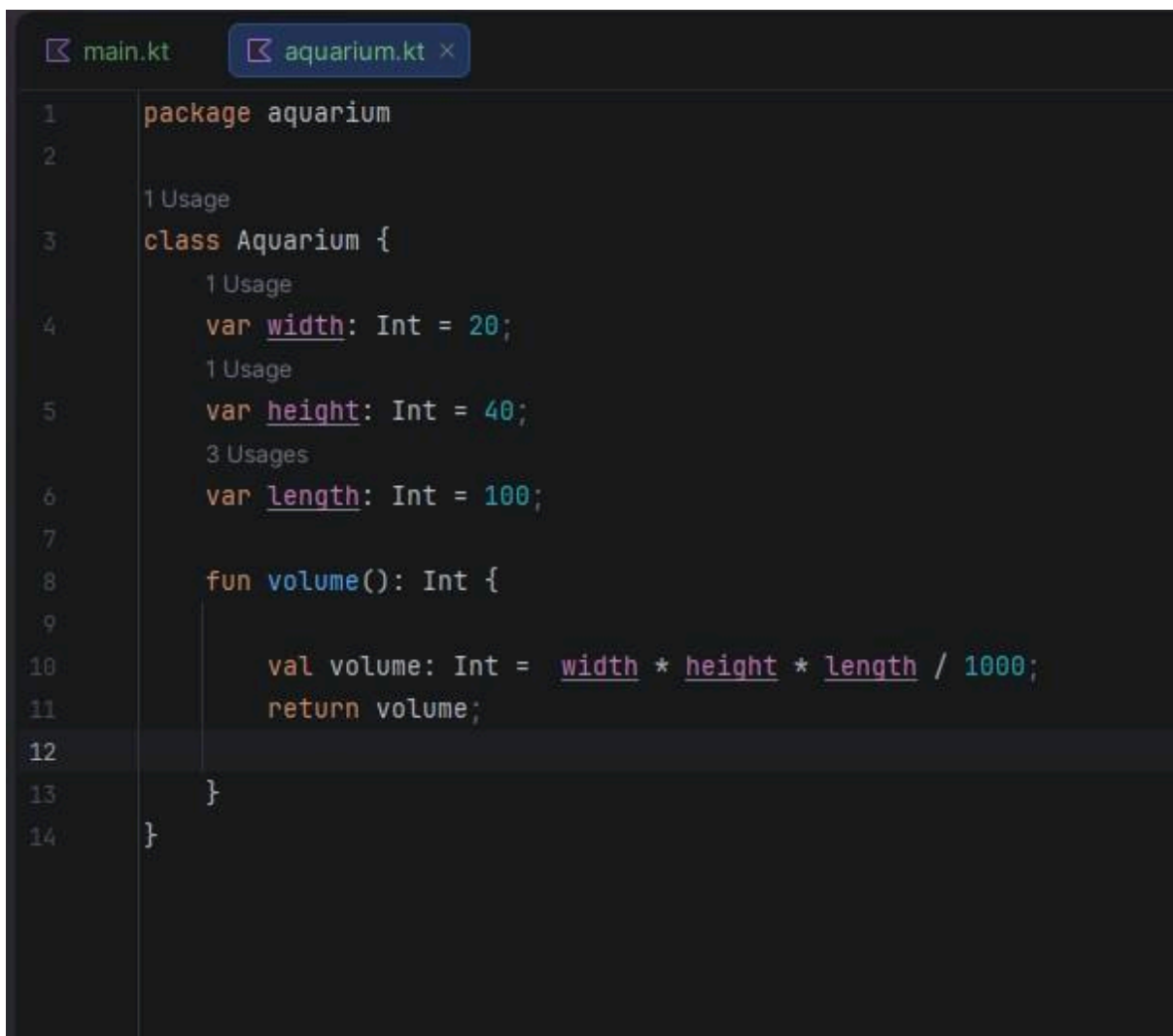
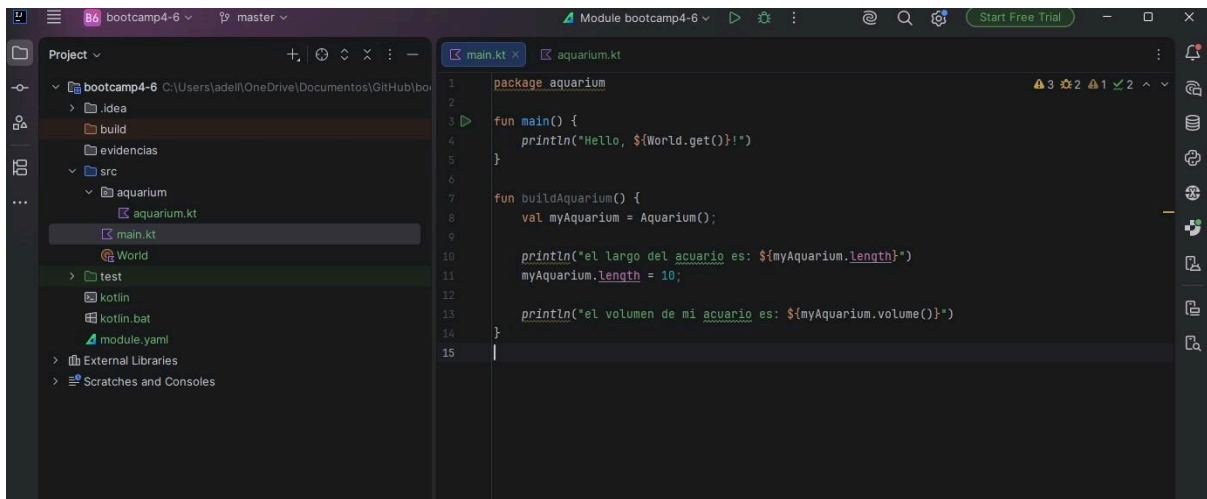
```

```

kotlin run --module bootcamp4-6 --main-class unidad6.DireccionesKt
✓ Compilation successful for 'bootcamp4-6' [jvm]
[START]
Game Over: [START, NORTH, SOUTH, EAST, WEST, END]
[]

Process finished with exit code 0

```




```
//3
1 Usage
fun <T : MaterialesBase> isSmallBuilding(building: Building<T>) {
    if (building.materialActualNecesitado < 500) {
        println("edificio pequeño")
    } else {
        println("edificio grande")
    }
}

fun main() {
    val woodBuilding = Building(material = Madera())
    woodBuilding.build()

    val brickBuilding = Building(material = Ladrillo())
    brickBuilding.build()

    isSmallBuilding(woodBuilding)
}
```

```
class Spice (val name: String="", val picor: String? = "mild"){ 9 Usages  Adel Mendez
    val heat: Int 1 Usage  Adel Mendez *
    get() = when (picor?.lowercase()) {
        "mild" -> 1
        "medium" -> 2
        "spicy" -> 3
        "very spicy" -> 4
        "extremely spicy" -> 5
        else -> 0
    }
    init {  Adel Mendez
        println("especia creada -> nombre $name, picor $picor, heat $heat")
    }
}

fun makeSalt() = Spice(name = "salt", picor= "mild"); 2 Usages  new *
```



```

kotlin run --module bootcamp4-6 --main-class MainKt
Hello, World!
la especias es curry con un pico de nivel medio y un heat de 5
-----
especia creada -> nombre Curry, picor medium, heat 2
especia creada -> nombre Pepper, picor mild, heat 1
especia creada -> nombre Cayenne, picor spicy, heat 3
especia creada -> nombre Ginger, picor mild, heat 1
especia creada -> nombre Red Curry, picor very spicy, heat 4
especia creada -> nombre Green Curry, picor medium, heat 2
especia creada -> nombre Habanero, picor extremely spicy, heat 5
especia creada -> nombre salt, picor mild, heat 1
-----
Leyendo 'Cien años de soledad': página actual -> 1
Leyendo 'Cien años de soledad': página actual -> 2
Leyendo eBook 'Clean Code' (texto): total palabras leídas -> 250
Leyendo eBook 'Clean Code' (texto): total palabras leídas -> 500
-----
Especia creada -> Nombre: Curry, Picante: medium, Heat: 3, Color: unidad4.YellowSpiceColor@53d8d10a (16776960)
Especia creada -> Nombre: Curry, Picante: spicy, Heat: 5, Color: unidad4.YellowSpiceColor@53d8d10a (16776960)
Moliendo Curry...
Nombre: Curry
Nivel de heat: 3
Color enum: YELLOW
Color RGB (decimal): 16776960
Color RGB (hexadecimal): #FFFF00
Curry es un tipo de Curry listo para moler.
Curry es un tipo de Curry listo para moler.

Process finished with exit code 0
|

```

```

package aquarium

import unidad4.simpleSpice

fun main() {
    println("Hello, ${World.get()}!")

    val especias = simpleSpice()
    println("la especias es ${especias.nombre} con un pico de nivel ${especias.picor} y un heat de ${especias.heat}")
}

fun buildAquarium() {
    val myAquarium = Aquarium()

    println("el largo del acuario es: ${myAquarium.length}")
    myAquarium.length = 10

    println("el volumen de mi acuario es: ${myAquarium.volume()}")
}

```

```

kotlin run --module bootcamp4-6 --main-class aquarium.MainKt
✓ Compilation successful for 'bootcamp4-6' [jvm]
Hello, World!
la especias es curry con un pico de nivel medio y un heat de 5

Process finished with exit code 0

```

```
// 2. Clase generica con
3 Usages.
class Building<out T : MaterialesBase>(val material: T) {

    1 Usage
    val MaterialBase: Int = 100

    2 Usages
    val materialActualNecesitado: Int
    |    get() = MaterialBase * material.cantidadNecesitada

    2 Usages
    fun build() {
    |    println("$materialActualNecesitado ${material::class.simpleName} necesitado")
    }
}
```

```
true
{Wilhelm Tell=Schiller, Jungle Book=Kipling, Hamlet=Shakespeare}

Process finished with exit code 0
```

```
class Libro(val title: String, val author: String, val year: Int) {

    1 Usage
    fun getTitleAuthor(): Pair<String, String> {
    |    return (title to author)
    }

    1 Usage
    fun getTitleAuthorYear(): Triple<String, String, Int> {
    |    return Triple(title, author, year)
    }
}
```

```
kotlin run --module bootcamp4-6 --main-class unidad5.BuildingKt
```

```
✓ Compilation successful for 'bootcamp4-6' [jvm]
```

```
400 Madera necesitado
```

```
800 Ladrillo necesitado
```

```
Construccion pequena
```

```
Process finished with exit code 0
```

```
|
```

```

1 Usage
fun libros1(){
    val libro = Libro( title = "Romeon and Juliet", author = "William Shakespeare", year = 1597)
    val libroAutor = libro.getTitleAuthor()
    val libroAutorAño = libro.getTitleAuthorYear()

    println("Here is your book ${libroAutor.first} by ${libroAutor.second}")

    println("Here is your book ${libroAutorAño.first} " +
        "by ${libroAutorAño.second} written in ${libroAutorAño.third}")
}

1 Usage
fun masLibros(){
    val allBooks = setOf("Macbeth", "Romeo and Juliet", "Hamlet", "A Midsummer Night's Dream")
    val library = mapOf("Shakespeare" to allBooks)
    println(library.any { it.value.contains("Hamlet") })
    val moreBooks = mutableMapOf<String, String>("Wilhelm Tell" to "Schiller")
    moreBooks.getOrPut( key = "Jungle Book") { "Kipling" }
    moreBooks.getOrPut( key = "Hamlet") { "Shakespeare" }
    println(moreBooks)
}

```

```

class Cachorro {
    1 Usage
    private val random = Random()

    1 Usage
    fun playWithBook(book: Libro) {
        // Genera un número aleatorio de páginas a romper entre 1 y 12
        val torn = random.nextInt( bound = 12) + 1
        book.tornPages(torn)
        println("El cachorro rompio $torn paginas, quedan ${book.pages} paginas (Peso: ${book.weight()} g)")
    }
}

1 Usage
fun librosYAsi(){

    val milibro = Libro( title = "Kotlin Programming", author = "JetBrains", year = 2023, pages = 50)
    val cachorro = Cachorro()
    println("Libro inicial: '${milibro.title}' con ${milibro.pages} paginas (Peso: ${milibro.weight()} g)\n")
    while (milibro.pages > 0) {
        cachorro.playWithBook(milibro)
    }

    println("\n paginas restantes: ${milibro.pages}")
}

```